

Kotlin循环嵌套与标签跳出教学讲义（零基础入门）

各位同学，今天咱们聚焦Kotlin中非常实用的“循环嵌套”和“标签跳出”。很多同学刚接触时会觉得“嵌套循环绕、跳出逻辑乱”，但只要跟着我用“生活场景+代码实操”的方式学，保证你10分钟就能搞懂！这两个知识点在Android开发中太常用了——比如遍历RecyclerView的二维数据、处理表格类信息都离不开它们。

一、先搞懂：什么是循环嵌套？

循环嵌套就是“循环里面套循环”，就像生活中“快递分拣”的场景：

- **外层循环**：遍历“快递区域”（比如北京、上海、广州三个区域）；
- **内层循环**：遍历每个区域里的“具体快递”（比如北京区域有3个快递，就要逐个处理）。

简单说：**外层循环控制“组”，内层循环控制“组内的每个元素”**，必须等内层循环执行完一轮，外层循环才会进入下一轮。

1. 基础案例：打印“区域-快递”清单

用代码模拟“3个区域，每个区域3个快递”的分拣场景：

Code block

```
1 fun main() {
2     // 外层循环：遍历3个区域（1-3号区域）
3     for (area in 1..3) {
4         println("开始处理 $area 号区域的快递")
5
6         // 内层循环：遍历每个区域的3个快递（1-3号快递）
7         for (express in 1..3) {
8             println(" $area 号区域 - $express 号快递：已分拣")
9         }
10
11         println("$area 号区域处理完成\n") // 内层执行完3次，外层才进下一轮
12     }
13 }
```

执行结果与逻辑分析

```

1  // 开始处理 1 号区域的快递
2      1 号区域 - 1 号快递：已分拣
3      1 号区域 - 2 号快递：已分拣
4      1 号区域 - 3 号快递：已分拣
5  1 号区域处理完成
6
7  // 开始处理 2 号区域的快递
8      2 号区域 - 1 号快递：已分拣
9      2 号区域 - 2 号快递：已分拣
10     2 号区域 - 3 号快递：已分拣
11  2 号区域处理完成
12
13 // 开始处理 3 号区域的快递
14     3 号区域 - 1 号快递：已分拣
15     3 号区域 - 2 号快递：已分拣
16     3 号区域 - 3 号快递：已分拣
17  3 号区域处理完成

```

核心逻辑：外层循环变量 `area` 每变一次（比如从1→2），内层循环变量 `express` 就要从1→3完整跑一遍——这就是“先内层后外层”的执行顺序。

2. Android开发场景：遍历二维列表数据

实际开发中，比如“课程表”就是典型的二维数据（行是星期，列是节次），必须用嵌套循环遍历：

Code block

```

1  fun main() {
2      // 二维列表：课程表（3行=3天，每行3列=3节课）
3      val courseTable = listOf(
4          listOf("数学", "语文", "英语"),    // 周一课程
5          listOf("物理", "化学", "生物"),    // 周二课程
6          listOf("历史", "地理", "政治")     // 周三课程
7      )
8
9      // 外层循环：遍历“天”（行）
10     for (day in courseTable.indices) { // indices获取索引（0=周一，1=周二，2=周三）
11         println("周${day+1}的课程：")
12
13         // 内层循环：遍历“当天的课”（列）
14         for (course in courseTable[day]) {
15             print("$course ")
16         }
17
18         println("\n")
19     }

```

执行后会清晰打印每天的课程，这就是嵌套循环在实际开发中的作用——处理“层级化数据”。

二、关键问题：嵌套循环如何精准跳出？

先看一个场景：在“快递分拣”时，一旦找到“紧急快递（比如3号区域2号快递）”，需要**立即停止所有分拣工作**（既停内层循环，也停外层循环）。这时候普通的 `break` 就不够用了——因为普通 `break` 只能跳出当前所在的内层循环，外层循环还会继续执行。

1. 普通break的“局限性”

我们给之前的快递案例加个需求：找到“3号区域2号快递”就停止。先用普通break试试：

Code block

```
1 fun main() {
2     for (area in 1..3) {
3         println("开始处理 $area 号区域的快递")
4
5         for (express in 1..3) {
6             if (area == 3 && express == 2) {
7                 println("找到紧急快递: $area 号区域 - $express 号快递, 停止分拣!")
8                 break // 普通break: 只跳出内层循环
9             }
10            println(" $area 号区域 - $express 号快递: 已分拣")
11        }
12
13        println("$area 号区域处理完成\n")
14    }
15 }
```

执行结果（问题所在）

Code block

```
1 开始处理 1 号区域的快递
2   1 号区域 - 1 号快递: 已分拣
3   1 号区域 - 2 号快递: 已分拣
4   1 号区域 - 3 号快递: 已分拣
5 1 号区域处理完成
6
7 开始处理 2 号区域的快递
8   2 号区域 - 1 号快递: 已分拣
9   2 号区域 - 2 号快递: 已分拣
```

```
10     2 号区域 - 3 号快递：已分拣
11     2 号区域处理完成
12
13     开始处理 3 号区域的快递
14     找到紧急快递：3 号区域 - 2 号快递，停止分拣！
15     3 号区域处理完成
```

问题：找到紧急快递后，内层循环停了，但外层循环还是执行完了3号区域的“处理完成”——我们要的是“立即停止所有循环”，这时候就需要**标签跳出了**。

2. 标签跳出：给循环“贴标签”，精准控制

标签跳出的核心思路：给“要跳出的循环”贴一个“标签”，然后用 `break@标签名` 的方式，直接跳出指定循环（哪怕是外层循环）。

语法规则：

- 标签格式： `标签名@` （比如 `outer@`、`mainLoop@`，名字随便起，见名知意即可）；
- 贴标签位置：在要控制的循环前面；
- 跳出方式： `break@标签名` （跳出标签对应的循环）。

修复案例：用标签跳出外层循环

给外层循环贴标签 `outer@`，找到紧急快递后用 `break@outer` 直接跳出外层循环：

Code block

```
1  fun main() {
2      // 给外层循环贴标签: outer@
3      outer@ for (area in 1..3) {
4          println("开始处理 $area 号区域的快递")
5
6          for (express in 1..3) {
7              if (area == 3 && express == 2) {
8                  println("找到紧急快递: $area 号区域 - $express 号快递，立即停止所有
分拣！")
9                  break@outer // 精准跳出标签为outer的外层循环
10             }
11             println(" $area 号区域 - $express 号快递：已分拣")
12         }
13
14         println("$area 号区域处理完成\n")
15     }
16
17     println("所有分拣工作已终止")
18 }
```

执行结果（符合预期）

Code block

```
1  开始处理 1 号区域的快递
2    1 号区域 - 1 号快递：已分拣
3    1 号区域 - 2 号快递：已分拣
4    1 号区域 - 3 号快递：已分拣
5  1 号区域处理完成
6
7  开始处理 2 号区域的快递
8    2 号区域 - 1 号快递：已分拣
9    2 号区域 - 2 号快递：已分拣
10   2 号区域 - 3 号快递：已分拣
11  2 号区域处理完成
12
13 开始处理 3 号区域的快递
14 找到紧急快递：3 号区域 - 2 号快递，立即停止所有分拣！
15 所有分拣工作已终止
```

完美！这就是标签跳出的核心价值——在嵌套循环中，精准控制要跳出的“目标循环”，避免逻辑混乱。

3. 标签跳出的其他用法：continue@标签

除了 `break@标签`，标签还能配合 `continue` 使用，意思是“跳过标签循环的当前轮次，直接进入下一轮”。

场景：快递分拣时，“2号区域”的快递暂时不处理，直接跳过该区域，继续处理3号区域：

Code block

```
1  fun main() {
2      outer@ for (area in 1..3) {
3          if (area == 2) {
4              println("2 号区域快递暂存，跳过该区域\n")
5              continue@outer // 跳过外层循环的当前轮次（2号区域），直接进3号区域
6          }
7
8          println("开始处理 $area 号区域的快递")
9          for (express in 1..3) {
10             println(" $area 号区域 - $express 号快递：已分拣")
11         }
12         println("$area 号区域处理完成\n")
13     }
```

执行后会直接跳过2号区域，这在需要“批量跳过某组数据”时非常实用。

三、Android开发实战：标签跳出的实际应用

在Android中，我们经常需要“遍历列表找目标数据，找到后立即停止”，比如从联系人列表中找到“手机号为138xxxx1234的联系人”：

Code block

```
1  // 数据类：联系人
2  data class Contact(val name: String, val phone: String)
3
4  fun findContact(contacts: List<Contact>, targetPhone: String): Contact? {
5      // 给外层循环贴标签
6      search@ for (contact in contacts) {
7          // 模拟联系人可能有多个手机号（内层循环）
8          val phones = listOf(contact.phone, contact.phone + "1", contact.phone
9              + "2")
10
11         for (phone in phones) {
12             if (phone == targetPhone) {
13                 println("找到目标联系人: ${contact.name}")
14                 return contact // 找到后直接返回，本质也是跳出所有循环
15                 // 若不需要返回，可用 break@search 跳出外层循环
16             }
17         }
18         println("未找到目标联系人")
19         return null
20     }
21
22     fun main() {
23         // 模拟联系人列表
24         val contacts = listOf(
25             Contact("小明", "1380000"),
26             Contact("小红", "1381234"),
27             Contact("小刚", "1390000")
28         )
29
30         // 查找手机号为13812341的联系人
31         findContact(contacts, "13812341")
32     }
```

这里用 `return` 直接返回实现了“跳出所有循环”，如果场景不允许返回，就可以用 `break@search` 精准跳出外层循环，逻辑完全一致。

四、总结与避坑指南

1. 核心知识点

- **循环嵌套**：外层控“组”，内层控“组内元素”，先执行完内层再执行外层；
- **普通break/continue**：只能控制当前所在的循环；
- **标签跳出**：给目标循环贴 `标签名@`，用 `break@标签名` / `continue@标签名` 精准控制。

2. 避坑指南

- 💡 1. 标签名可以随便起，但建议用“outer”“mainLoop”等有意义的名字，避免和变量名重复；
- 2. 不要滥用标签：只有“需要跳出外层循环”时才用，普通场景用普通break即可；
- 3. 若找到目标后需要返回数据，优先用 `return`（更简洁），其次再用标签跳出。

3. 课后作业：实战练习

需求：模拟“学生成绩表”，用嵌套循环实现以下功能：

1. 定义二维列表：3个班级（1-3班），每个班级4个学生的成绩（比如1班：[85,92,78,100]）；
2. 遍历成绩表，计算每个班级的平均分并打印；
3. 一旦发现“有学生成绩满分（100分）”，立即停止所有计算，打印“发现满分学生，停止统计”。

提示：用标签跳出外层循环，满分判断条件写在内层循环中。

（注：文档部分内容可能由 AI 生成）